Efficient Pod IP Allocation and CNI Resilience

Arpan Gupta
24110051

Buha Deep Maheshbhai
24110082

Param Tanna
24110236

Ramji Purwar
24110287

Rathod Yuvraj Rajubhai
24110293

Solanki Viraj Rajeshbhai
24110348

I. Introduction

Kubernetes runs most of today's cloud applications. Every pod gets its own IP address, and any pod must be able to reach any other pod without NAT. Kubernetes does not build this network itself. It hands the job to a ***CNI plugin****, such as Calico or Flannel. The CNI creates the pod's network interface, and its* ***IPAM*** *part decides which IP address the pod gets.*

In this project we look at two design weaknesses in how pod networking itself works. One is efficiency, the other is resiliency.

1. ***Efficiency (IPAM).*** *Kubernetes gives every node a fixed-size block of pod addresses at creation time (by default a /24, i.e. 256 addresses). Quiet nodes waste most of their block, while busy nodes run out and their new pods cannot start, even though the cluster as a whole has enough free addresses.*

2. ***Resilience (CNI).*** *Every pod has exactly one network interface from one CNI. If that CNI's data path breaks, the pod is unreachable, and Kubernetes usually does not notice, because the node still reports Ready.*

Our approach. *For the first problem we built a* ***buddy-style IPAM controller****. It works like the buddy memory allocator in an operating system [1]: it hands out power-of-two, aligned address blocks to each node, grows a node before it runs out, and takes blocks back when the node becomes quiet. For the second problem we give each pod* ***two network interfaces from two independent CNIs*** *(Calico as primary, Flannel as backup, joined by Multus). A health agent on every node probes both paths, and our operator moves Service traffic to the backup path when the primary path of a node is proven broken, then moves it back when the primary is repaired.*

II. Problem Statement

This project looks at two networking problems in Kubernetes. The first is about how pod IP addresses are given to nodes. The second is about what happens to pod traffic when the pod network on one node breaks.

Problem 1: Static pod IP allocation wastes addresses and starves busy nodes

Every pod needs an IP address. Kubernetes splits the cluster's pod address range into equal parts, one part per node (by default a /24, which is 256 addresses). This part is fixed when the node joins and never changes, even when one node runs only a few pods and another runs many.

For example, Node A runs 100 pods and node B needs 500. Both get 256 addresses. Node A leaves 156 addresses unused, while 244 pods on node B cannot start. The scheduler does not track IP addresses, so these pods stay stuck in `ContainerCreating`. If each node got a block sized to its need (a /25 with 128 addresses for A and a /23 with 512 for B), every pod would get an address.

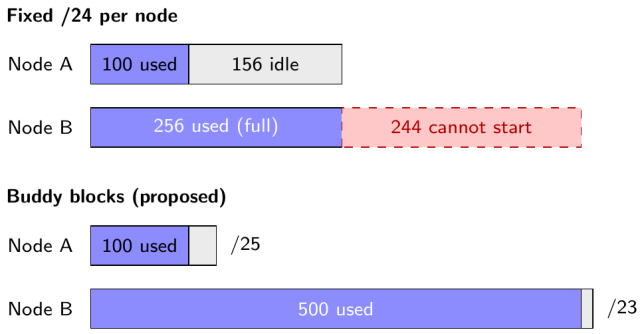


Fig. 1. Example with a fixed /24 per node, node A leaves 156 addresses idle while 244 of node B's pods cannot start. Buddy blocks give A a /25 and B a /23.

The fixed split causes four problems:

1. ***Waste and starvation****: quiet nodes keep addresses they do not use, while busy nodes run out.*
2. ***No resizing****: a node's range cannot grow or shrink while the node is running.*
3. ***No contiguous growth****: the addresses next to a full range are usually used by something else, so extra space must come from a separate CIDR.*
4. ***Shrinking is blocked****: a block cannot be taken back while even one pod is still using an address in it.*

Existing tools only solve part of this. Calico gives out small 64-address blocks on demand, but when a node has no free block it borrows single addresses, and each borrowed address needs its own /32 route.

No current solution gives each node address space that grows and shrinks with its demand while still keeping one route per aligned block.

Problem 2: A pod's single CNI is a single point of failure

A pod normally has one network interface, one IP address and one path, all set up by one CNI plugin. If that path breaks, the pod cannot be reached, even though the application inside it is healthy.

A CNI has several parts: the plugin binary, a node agent (such as `calico-node`), the IP store and the data path. Any of them can fail. When the data path fails, Kubernetes does not notice:

- ***The node stays Ready**: the kubelet talks to the API server over the host network, not the pod network, so its heartbeats still arrive (Figure 2).*
- ***Pods are not moved**: Kubernetes only moves pods when a node is `NotReady`.*
- ***Restarts do not help**: restarting the pod's containers does not repair the network.*
- ***The fix is manual**: an operator has to restart the CNI or reboot the node.*

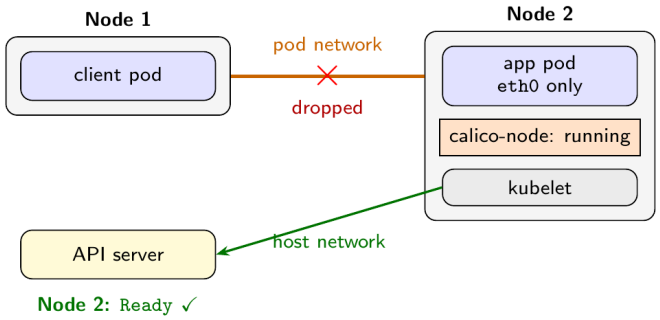


Fig. 2. The pod network to node 2 is broken and requests are dropped, but the kubelet still reaches the API server over the host network, so Kubernetes keeps node 2 Ready.

Multus [4] can already give a pod more than one interface, but it is used for performance, not as a backup path.

III. Project Objectives

IPAM (Problem 1)

1. *Serve 100% of pod demand that fits in the pool, in all test workloads, including skewed and bursty load.*
2. *Hold fewer addresses per node than a fixed /24 when load is light or uneven.*
3. *Grow a node **before** it runs out, and grow it immediately when a pod fails with "no IP addresses available".*
4. *Support discontiguous CIDRs, safe shrinking (no running pod loses its address) and reclaiming of leaked addresses.*

CNI resilience (Problem 2)

1. *Detects a broken primary path on one node within about 30 seconds, without false alarms from a single lost packet.*
2. *Never blame a node unless the probe pattern proves it, take no action when the pattern is ambiguous.*
3. *Move Service traffic of the broken node to the backup interface automatically, so client requests succeed again.*
4. *Move traffic back automatically, with no failed request, once the primary path works.*

V. Architecture and Design

Buddy IPAM

*The design has two parts (Figure 3): a **controller** that decides which blocks each node owns, and a **node agent** that makes this true on the host.*

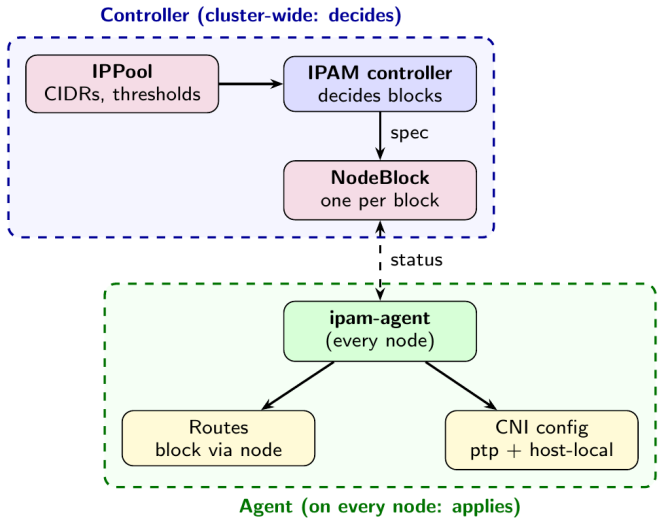


Fig. 3. Buddy IPAM architecture. The controller (cluster-wide) writes NodeBlock objects; the agent on every node installs routes and writes the CNI config, and reports back through the NodeBlock status.

***Custom resources.** IPPool holds the ordered list of CIDRs and the tuning values (grow threshold 80%, growth 50%, shrink threshold 50%, minimum block /28). NodeBlock is one allocated block. Its name comes from its CIDR, so the API server's "names must be unique" rule makes it impossible to give the same block out twice.*

Buddy allocation. *Each pool CIDR is a binary tree of blocks: a /22 splits into two /23 "buddies", each /23 into two /24s, down to /28 (16 addresses). A request takes the smallest free block that fits. When a block and its buddy are both free they merge back. Every block stays aligned, so it is always one routable prefix.*

Controller loop. *Each pass rebuilds the whole buddy tree from the existing NodeBlocks, so the controller keeps no memory and a restart changes nothing. For every node it then:*
- gives a new node a first block of about 25% of its max pods.
- grows the node when free addresses fall below 20% of its capacity, or when a kubelet event says a pod failed with "no IP addresses available".
- otherwise checks whether a block can be given back (shrink).

Block lifecycle. *A new block is Pending until every other node has installed a route to it, and only then becomes Active. This order (routes, then config, then Active) ensures a pod never gets an address that nobody can reach. To shrink, the controller marks a block Draining: the agent removes it from the CNI config, so no new pod uses it, but existing pods keep their addresses. When the last pod ends, the block becomes Quarantined for 60 s (so late packets cannot reach a new owner) and then Released. If the node needs the space again while the block is still Draining, the block goes back to Active.*

Node agent. *A privileged DaemonSet on every node. Each pass it (1) installs block via <owner node IP> routes for other nodes' blocks, merging buddy blocks into one prefix, (2) writes the CNI config with all Active blocks of this node, (3) counts host-local lease files to report real usage, and (4) deletes leases older than 2 minutes that no pod holds (leak reclaim).*

Dual-CNI Resilience

Figure 4 shows the setup. Every application pod has two interfaces: eth0 from ***Calico*** *(primary, IPIP tunnel, 172.16.0.0/16) and net1 from* ***Flannel*** *(backup, VXLAN tunnel, 10.244.0.0/16), added by Multus. The two share no agent, address range or tunnel, so one can fail while the other works.*

Step 1 - Detect (health agent). *A DaemonSet agent on each node sends HTTP probes to agents on other nodes over each network separately: a small request and a large one (1400 bytes) to also catch MTU problems. Agents probe only each other on port 9100, which a NetworkPolicy allows, so an application's NetworkPolicy can never look like a network failure. Each node probes one peer per round in a rotating ring (node* ***i*** *probes node* ***(i + r) mod N****), so a round costs N probes (as in Pingmesh [5]). A path changes state only after* ***3 identical results in a row*** *(probe every 2 s), like BFD [6]. Results are written as an annotation on the agent's own Node.*

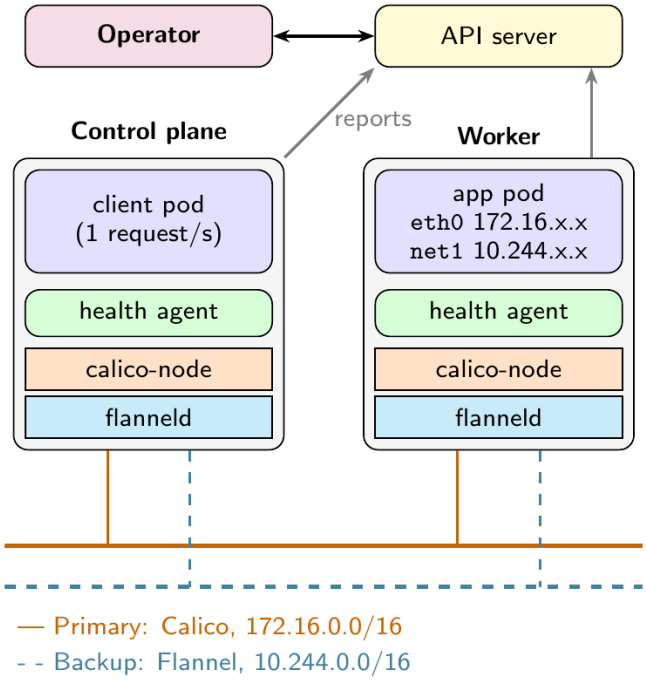


Fig. 4. Dual-CNI design (two nodes shown). Each pod has a Calico and a Flannel interface. Health agents probe each other over both networks and report to the API server, where the operator reads the reports and acts.

Step 2 - Decide. *A failed probe from A to B could mean A, B or the link between them is broken. A node is blamed only if all probes* ***into*** *it failed, all probes* ***out of*** *it failed (or it went silent),* ***and*** *some probe between two* ***other*** *nodes worked. Anything else is ambiguous, and the operator does nothing: a wrong failover is worse than a late one.*

Step 3 - Fail over. *Services only know a pod's primary IP. We change this with two standard Kubernetes features:*

- ***Readiness gate****: a custom readiness condition controlled by our operator. Turning it off makes Kubernetes itself remove the eth0 address from the Service.*
- ***Shadow EndpointSlice****: an extra EndpointSlice for the same Service listing the pod's net1 address, so kube-proxy sends requests over the backup network.*

This is the "active/standby" option, like VRRP [7]. If the backup is also broken, the operator ***isolates*** *the node: its pods leave the Service and traffic goes to healthy replicas. If the primary is not proven broken but some probe through the node still fails, the operator keeps the current state and waits.*

Step 4 - Recover and fail back. *On a new fault the operator taints the node (NoSchedule, so no new pods land there) and restarts its calico-node pod* ***once****. When every probe touching the node works again, it turns the gate back on* ***before*** *deleting the shadow slice (so the Service is never empty) and removes the taint.*

VI. Implementation

Languages and frameworks. *Both parts are written in* ***Go*** *using* ***Kubebuilder*** *and* ***controller-runtime*** *[8]. Routes are installed with the netlink library. The dashboard is a React + TypeScript app (Vite). Unit tests cover the buddy allocator, planner, agent, blame rule and failover logic, and run in GitHub Actions CI.*

Code repository*: https://github.com/yuvraj-rathod-1202/CS331-CN-Project-1 (operator code in operator/, IPAM in internal/controller/ipam/, CNI resilience in internal/controller/cniresilience/).*

IPAM: Grow decision (pseudocode).

```
for each node n (sorted):
  if n has no block:
    allocate first block (≈25% of max pods, min /28)
  elif a block is Pending: wait (one change in flight per node)
  elif capacity(n) - demand(n) < threshold or starved(n):
    if n has a Draining block: restore it
    elif n has a Releasing half:    unrelease it
    else:
      shortfall = requiredCapacity(demand) - capacity
      allocate shortfall from buddy of merged range
  elif n can give a block back: mark emptiest block Draining
  elif n has grown block with empty half: set spec.release to
empty half
```

CNI: Blame rule (pseudocode).

```
for each node x:
  in  = probes into x;  out = probes from x
  if all(in failed) and (all(out failed) or x is silent)
    and some probe between two other nodes worked:
    x is broken
remaining failed probes between nodes seen working
elsewhere –> broken link
anything else –> ambiguous
```

Challenges and how we solved them.

- ***Address clash between CNIs.*** *Flannel can only use the nodes' 10.244.x.0/24 ranges, so we moved Calico to 172.16.0.0/16.*
- ***Editing EndpointSlices did not last.*** *Kubernetes rewrote them within a second, so we used readiness gates plus separate shadow slices.*
- ***Restart loops.*** *A calico-node restart never fixed a data-path fault in our tests and briefly takes the node's network down, so we restart it only once per fault.*

VII. Tested Setup

All experiments ran on a single laptop with an Intel Core i5 and 16 GB RAM, running Windows 11 with Docker Desktop under WSL 2. Kubernetes clusters were created with minikube v1.37.0 using the Docker driver, each with 3 nodes (1 control-plane and 2 workers) on Kubernetes v1.37. The CNI resilience cluster (`k8s-experiments`) used Calico v3.32.2 as the primary CNI (IPIP tunnel, `172.16.0.0/16`), Flannel v0.26.7 as the backup (VXLAN tunnel, `10.244.0.0/16`), and Multus to attach both interfaces to every pod. The operator and agents were built with Go 1.26 and the live dashboard with Node.js 18+, kubectl was used for cluster interaction throughout.

IPAM settings. *Pool 10.244.0.0/22 plus discontiguous spill CIDR 10.200.0.0/23, blocks /28 to /22; grow at 80% used, grow by 50%, cap 120% of max pods, shrink when ≤ 50% used, 300 s cooldown, 60 s quarantine.*

CNI workload. *resilience-target (2 nginx replicas, one per worker, each with `net1` and the readiness gate) behind resilience-service. A traffic-client pod on the control-plane node sends one request per second; its log is our measurement. CNIResilience settings: probe every 2 s, 1 s timeout, 3 failures to change state, auto-failover and auto-recover on. Faults are injected with tagged iptables DROP rules (e.g. drop the worker's pod traffic, drop IPIP protocol 4, drop Flannel VXLAN) so a "Reset" removes them all.*

VIII. Evaluation

We evaluate our solution through two experiments designed around the two core problems:

1. ***IPAM Efficiency Experiment****: Testing hundreds of nodes and thousands of pods exceeds a single laptop's resources. We therefore built a discrete-event simulator that runs our real controller planning code. We simulated clusters of 40 to 1,000 nodes under five traffic patterns and compared buddy allocation against Kubernetes' default static /24, a static /25, and Calico on-demand blocks.*

2. ***CNI Resilience Experiment****: We set up a live 3-node minikube cluster where every application pod has two network interfaces: Calico as the primary path and Flannel as the backup. A client pod sends continuous HTTP requests (one per second) while we inject 10 realistic network faults (e.g., dropping pod traffic, breaking tunnel protocols, killing CNI agents, or pausing nodes) to measure detection speed, failover reliability, and zero-loss recovery.*

IX. Summary

Kubernetes gives nodes fixed address blocks that are wasted on quiet nodes and run out on busy ones, and each pod depends on one CNI path that can break unnoticed. Our buddy IPAM grows and shrinks each node's aligned blocks with demand; in simulation it served 100% of pods in every workload and up to 1000 nodes (a fixed /24 served as little as 26%), with 2-3× better utilisation than a /24 and 1-3 routes per node. Our dual-CNI design detected a broken primary

path in 10-31 s on a real cluster, moved all traffic to the Flannel backup, and moved it back with no failed request.

X. Reproducibility

GitHub:https://github.com/yuvraj-rathod-1202/CS331-CN-Project-1

Requirements: Docker, minikube, kubectl, Go, Node.js. All commands start from the repository root. Full step-by-step guides are in operator/docs/ipam-evaluation.md, experiments/cni-resilience-experiment.md, and operator/docs/ipam-agent.md.

References

*[1] K. C. Knowlton, "A fast storage allocator," *Communications of the ACM*, vol. 8, no. 10, pp. 623–625, Oct. 1965, doi: 10.1145/365628.365655.*

[2] Kubernetes SIG Network, "KEP-2593: Enhanced NodeIPAM to support discontiguous cluster CIDR," Kubernetes Enhancement Proposals, withdrawn in v1.29. [Online]. Available: https://www.kubernetes.dev/resources/keps/2593/

[3] Google Cloud, "Discontiguous multi-Pod CIDR," GKE documentation. [Online]. Available: https://cloud.google.com/kubernetes-engine/docs/how-to/multi-pod-cidr

[4] Kubernetes Network Plumbing Working Group, "Multus CNI." [Online]. Available: https://github.com/k8snetworkplumbingwg/multus-cni

*[5] C. Guo *et al.*, "Pingmesh: A large-scale system for data center network latency measurement and analysis," in *Proc. ACM SIGCOMM*, London, UK, 2015, pp. 139–152.*

[6] D. Katz and D. Ward, "Bidirectional Forwarding Detection (BFD)," RFC 5880, IETF, Jun. 2010.

[7] S. Nadas, "Virtual Router Redundancy Protocol (VRRP) Version 3 for IPv4 and IPv6," RFC 5798, IETF, Mar. 2010.

[8] Kubernetes SIGs, "Kubebuilder and controller-runtime." [Online]. Available: https://book.kubebuilder.io

[9] Tigera, "Calico documentation: IP address management and IP pools." [Online]. Available: https://docs.tigera.io/calico/latest/networking/ipam/

[10] Flannel project, "Flannel: a network fabric for containers." [Online]. Available: https://github.com/flannel-io/flannel

[11] CNCF, "Container Network Interface (CNI) specification." [Online]. Available: https://github.com/containernetworking/cni/blob/main/SPEC.md

[12] The Kubernetes Authors, "Pod readiness gates" and "EndpointSlices," Kubernetes documentation. [Online]. Available: https://kubernetes.io/docs/

Appendix A: Team Work Distribution and Member Rating

A.1 Work Distribution

#	Member Name	Roll No.	Tasks Done	Rating (1–10, unique)
1	Arpan Gupta	24110051	CNI operator debugging	8
2	Buha Deep Maheshbhai	24110082	CNI operator bug fixes, initial web app draft	9
3	Param Tanna	24110236	Web app review	5
4	Ramji Purwar	24110287	Buddy IPAM controller, IPAM web demo.	7
5	Rathod Yuvraj Rajubhai	24110293	Problem formulation, CNI resiliency operator code, report & slides.	10
6	Solanki Viraj Rajeshbhai	24110348	Report preparation	6

Appendix B: AI Usage Details

B.1 Prompts

#	AI Tool	Purpose
1	ChatGPT	Problem statement explorations and ideations
2	Claude Chat	Problem statement document preparation and research purposes
3	Antigravity	Code generation, document & slide preparation
4	Claude Code	Code generation, solution understanding, research and debugging purposes